

Tecnun
Universidad
de Navarra

Abstract

Splitting shared expenses fairly among a group of people is a common problem, solved in practice by apps like Tricount using simple greedy algorithms. While fast, these methods do not **guarantee** the minimum number of transactions needed to settle all debts. In this work, we model the debt settlement problem as a mixed integer linear program (MILP) that minimizes the total number of transfers while guaranteeing exact settlement of all balances. We present the full mathematical formulation, discuss the role of binary variables and the Big-M constraint, utilizing the MILP approach. We also discuss computational complexity and practical extensions of the model, discussing as well the comparison of this model to others, such as greedy and quadratic.

1: Introduction

1.1 Motivation and scope

Splitting expenses is something most people deal with at some point: whether living with roommates, going on a group trip, or just covering each other for food and drinks. The scenarios are endless: one person books the hotel on their card, another does the grocery run for the whole flat, someone floats a friend for a concert ticket. Individually, each of these is simple enough to track. But as the group grows and time passes, the web of who owes what to whom becomes genuinely difficult to manage.

The traditional solution, everyone just remembers what they owe, falls apart quickly in practice. People forget, amounts get mixed up, and what should be a straightforward settlement turns into an awkward conversation weeks later. Even when people try to be careful, keeping a mental or physical record of debts across a group of five or ten people over a month-long trip is error-prone and unnecessarily stressful.

This is exactly the gap that apps like Tricount and Splitwise fill. Rather than asking each person to track individual debts, these tools aggregate all expenses, compute each person's net balance, and then work out a simplified set of payments that settles everything. The key insight is that you rarely need everyone to pay everyone else back directly. If person A owes person B €5, and person B owes person C €5, then A can simply pay C directly and the whole chain collapses into a single transaction. Generalizing this idea across a larger group is where mathematics comes in.

The underlying problem: **finding the smallest possible set of transactions that settles all debts exactly**, turns out to be a genuine optimization problem, and a complex one at that. With this in mind, the objective of this project is to model and solve this problem formally, using the optimization tools covered in class. Specifically, we formulate it as a mixed integer linear

program (MILP) that minimizes the total number of transactions while guaranteeing that every balance is settled exactly.

1.2 Problem Statement

The problem statement that then comes to mind is: **Given a group of people who have been sharing expenses, each with a known net balance (either positive (they are owed money) or negative (they owe money)) how can we settle all debts in a way that minimizes the total number of transactions required?**

This might sound straightforward, but it is more complex than thought. The number of possible ways to arrange payments across even a modest group grows rapidly, and finding the arrangement with the fewest transactions is not something that can be done reliably by intuition or simple rules. As we will discuss later in the report, this problem belongs to a class of combinatorial optimization problems for which no efficient general solution is known.

With this in mind, the goal of this project is to construct a mathematical model that solves this problem exactly. Specifically, we want a model that:

- takes as input the net balance of each participant,
- determines who should pay whom and how much,
- guarantees that every balance is settled in full,
- and does all of this while minimizing the total number of transactions.

Rather than **approximating** a solution the way real apps do, we want to find the actual optimal one. This leads us to the framework of mixed integer linear programming, which is powerful enough to capture both the continuous amounts being transferred and the discrete decision of whether a transaction between two people takes place at all.

1.3 Structure

This report approaches the problem from the ground up, starting from the real-world context and building toward a formal mathematical solution.

We begin in **Section 2** by looking at how apps like Tricount actually solve this problem in practice. Rather than exact optimization, these systems rely on greedy heuristics: fast, simple algorithms that produce a good solution without guaranteeing the best one. We explain how these heuristics work, walk through an example, and discuss why real applications prefer them despite their limitations.

Section 3 provides the theoretical backdrop that motivates the use of exact optimization in the first place. We examine why minimizing the number of transactions is computationally hard, connecting the problem to known results in complexity theory, and discuss why exact methods remain practical for the group sizes typically seen in shared expense applications.

The core of the report is **Section 4**, where we develop the full MILP formulation. This includes the classification of participants into debtors and creditors, the definition of decision variables, the objective function, and each constraint in detail. We pay particular attention to the role of binary variables and the Big-M linking constraint, which are the key features that make this a mixed integer program rather than a standard linear one.

Section 5 then applies the model to a concrete worked example with a small group of participants, solving it step by step.

In Section 6 we discuss the comparison of the MILP against alternative optimization models including a linear programming, a quadratic programming variant, and the heuristic method used by the apps. **We discuss** what each approach optimizes, what it gives up, and when it might be preferred.

Finally, **Section 7** discusses possible improvements and extensions to the base model, **Section 8** draws conclusions, and the appendix contains the solver code used to verify the numerical example.

2: How real systems solve this problem: Greedy

2.1 Explanation and mapping

The traditional way people mentally manage transactions is that if there are N number of transactions, then the group has to make N number of payments. Splitwise and Tricount aim to minimize this number of transactions in order to reduce complexity and time spent in banking apps.

The debt settlement can be visually mapped. Imagine drawing a diagram where every person in the group is represented by a node, and every time someone owes money to someone else, you draw an arrow between them pointing from the person who owes to the person who is owed, with the amount written on it. So if Alex owes Maria €10, there is an arrow from Alex to Maria labelled €10. Do this for every debt in the group and you get a map of who owes what to whom, called a directed weighted graph. The arrows have a specific direction, and are weighted because each arrow carries a numerical value.

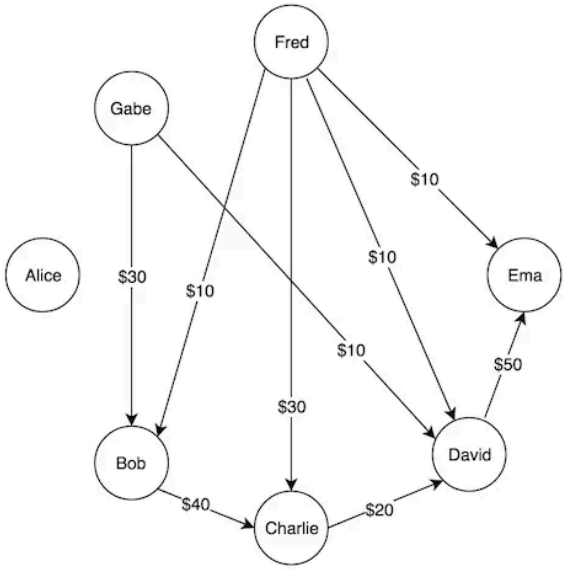


Figure 1: Original mapping of transactions.
Source: [Mithun Mohan K (2019)]

With this representation in place, the debt settlement problem can be stated cleanly: given this map of debts, can we redraw it with **fewer arrows** while ensuring that every person's net balance remains exactly the same? The redrawn map would have fewer transactions but would be financially equivalent to the original. This is precisely what apps like Splitwise and Tricount are trying to compute.

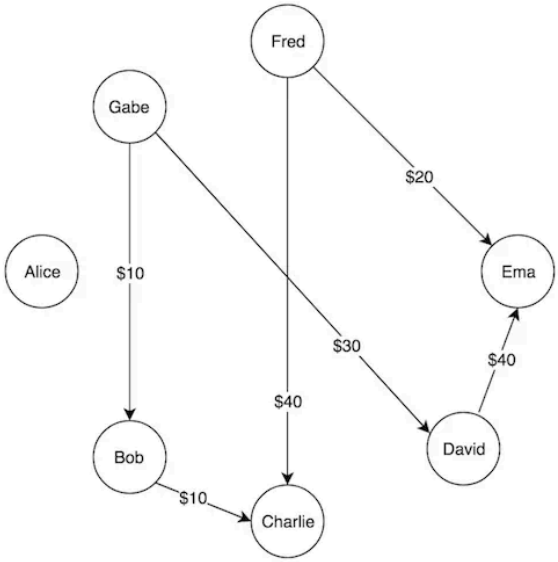


Figure 2: Mapping after algorithm
Source: [Mithun Mohan K (2019)]

This belongs to a class of problems known as **minimum cost flow problems**, where the objective is to find the most efficient way to route quantities through a network. In our case, the "quantity" is money, the network is the group of people, and the "cost" we are minimizing is not the total amount transferred but simply the number of individual transactions that need to take place.

2.2 The greedy algorithm: how it works and where it falls short

The approach that apps like Tricount and Splitwise use is known as a **greedy algorithm**. This is how it works: It repeatedly resolves the largest outstanding debt first. At each step, it looks at everyone's current balance, finds the person who owes the most (the biggest debtor) and the person who is owed the most (the biggest creditor), and has the debtor pay the creditor as much as possible in a single transaction. This either clears the debtor's balance entirely, clears the creditor's balance entirely, or both. Then the balances are updated and the process repeats with whoever still has an outstanding balance.

A small example makes this clearer. Suppose we have three people with the following net balances after all expenses:

Alex owes €15
Maria is owed €10
Juan is owed €5

Step 1: The biggest debtor is Alex (−€15) and the biggest creditor is Maria (+€10). Alex pays Maria €10, which clears Maria's balance entirely. Alex's remaining balance is now −€5.

Step 2: The biggest debtor is now Alex (−€5) and the only remaining creditor is Juan (+€5). Alex pays Juan €5, which clears both balances simultaneously.
Result: 2 transactions and everyone is settled.

In this case greed happens to find the optimal solution, but that is not always guaranteed. In more complex scenarios with more people, the greedy matching at one step **can prevent a better combination from being found later**, resulting in more transactions than the true minimum. This is the limitation of the greedy method.

It makes the best local decision at each step without considering whether a different combination of payments might settle everything in fewer moves overall. Finding that truly optimal combination requires checking which subsets of debtors add up exactly to each creditor's total. This grows exponentially with the number of people in the group and becomes computationally inviable. This is mathematically equivalent to the Subset Sum Problem, a well known NP-complete problem, meaning there is no known algorithm that can solve it efficiently for large inputs, rather it has to go one by one checking all combinations in no particular set order. There is no shortcut to find the guaranteed best answer.

For a group of five or six people this is not an issue. But as the group grows, the number of combinations to check becomes computationally infeasible, which is precisely why these apps accept the greedy solution. It is fast, fair enough in practice, and works well for the vast majority of real-world group sizes. Greedy has a time complexity $O(n^2)$ and a space complexity of $O(n)$ making it suitable for these apps

3: Our approach: Mixed integer linear programming

3.1 The gap our model fills

To understand why a more rigorous method is needed, consider the following example where the greedy algorithm falls short. Suppose five participants have the following net balances:

- Debtors: Alex (−11), Sofia (−8), Laura (−5)
- Creditors: Maria (+13), Juan (+11)

The greedy solution. The greedy algorithm always pairs the largest debtor with the largest creditor.

Step 1: The biggest debtor is Alex (€−11) and the biggest creditor is Maria (€+13). Alex pays Maria €11. Alex is cleared, Maria still needs €2.

Step 2: The next biggest debtor is Sofia (€−8) and the biggest creditor is Juan (€+11). Sofia pays Juan €8. Sofia is cleared, Juan still needs €3.

Step 3: The only remaining debtor is Laura (€−5). She first pays Maria the remaining €2, then pays Juan the remaining €3.

Total transactions: 4

The optimal solution. The MILP model looks at the whole picture and identifies a perfect subset match that greedy missed.

Step 1: Alex (€−11) pays Juan (€+11) directly. Both are cleared in a single move.

Step 2: Sofia (€−8) pays Maria (€+8).

Step 3: Laura (€−5) pays Maria (€+5). Maria is cleared (8 + 5 = 13).

Total transactions: 3

Greedy has no way of knowing whether its current match is blocking a better arrangement later on. It just picks the biggest pair and moves forward blindly, picking a local optimum. This is precisely the gap our model addresses. Rather than making one local decision at a time, we want a method that considers all possible combinations simultaneously and finds the arrangement with the provably minimum number of transactions. That is what Mixed Integer Linear Programming allows us to do.

Beyond the mathematical motivation, the reason we chose this approach is also simpler: we wanted to take a problem we actually encounter in everyday life and see if we could apply the optimization methods learned in class to solve it in our own way. The fact that it works, and works better than what these apps currently do for small groups, is what makes it interesting.

3.2 Explanation on MILP

Mixed Integer Linear Programming is an extension of Linear Programming (LP) that allows variables to be restricted to integer values. In standard Linear Programming, all variables are continuous. Since our objective is to minimize the number of transactions, we must introduce logical "switches" that can only be represented by integers, specifically, binary variables.

The true advantage of MILP with debt settlement lies in its ability to handle two distinct types of data simultaneously.

3.2.1 Continuous vs. Binary Variables

Our model utilizes two types of decision variables to represent the state of the settlement:

1. **Integer Variable** (x_{ij})**:** This represents the actual amount of money transferred from debtor i to creditor j. Rather than working with decimals, we work in whole cents so €40.00 becomes 4000. This means the transfer amounts are always whole numbers, which keeps the values cleaner than working with floating points and avoids rounding errors in the solver.
2. **Binary Variables** (y_{ij}): In this model, $y_{ij} = 1$ signifies that a transaction occurs from person i to person j. $y_{ij} = 0$ signifies that no money is exchanged.

By setting the objective function to minimize the sum of all binary variables (), we force the solver to find a solution that uses the fewest "active" connections possible, regardless of the total volume of money being moved.

3.3 Computational complexity

The trade-off for finding this true minimum is computational cost. Unlike greedy which runs in $O(n^2)$, MILP is NP-hard, meaning in the worst case the solver has to explore an exponential number of $O(2^{pq})$ combinations where p is the number of debtors and q the number of creditors. For a group of 1000 people this would be completely infeasible. However, for the group sizes that shared expense apps actually deal with, typically between 5 and 20 people, the solver may take some time however is able to eventually solve it in a reasonable time. The complexity is expensive, but in theory the payoff is a guaranteed optimal solution.

Beyond the mathematical motivation, the reason we chose this approach is also simpler: we wanted to take a problem we actually encounter in everyday life and see if we could apply the optimization methods learned in class to solve it in our own way. The fact that it works, and works better than what these apps currently do for small groups, is what makes it interesting.

4: The Model

4.1: Input and balance computation

The starting point of our model is simple: each person in the group reports how much they have spent in total on shared expenses. This ends up being a list of expenditures, one per person. From this, the model computes everything else automatically, by receiving a vector that's length is determined by the amount of people.

The first step is determining how much each person should have paid in a perfectly fair split. The total expenditure across the group is divided equally among all n participants, giving a base fair share. Since we work in integer cents, representing €40.00 as 4000, €7.50 as 750, and so on, the division does not always come out exactly. To handle this cleanly, the base share is taken as the floor of the total divided by n, and any remaining cents are distributed one at a time to the first participants. This ensures the fair shares always sum to exactly the total expenditure with no rounding errors.

```
matlab

parte_base = floor(total/nump);

resto = mod(total,nump);
```

```
reparto = parte_base * ones(1,nump);

reparto(1:resto) = reparto(1:resto) + 1;
```

Once the fair share is established, the net balance of each person k is computed as:

$$s_k = \text{amount spent by } k - \text{fair share of } k$$

A positive balance means the person spent more than their share and is owed money, they are a **creditor**. A negative balance means they spent less than their share and owe money, they are a **debtor**. A balance of zero means they already paid exactly their share and no action is required.

We define the set of debtors as: $D = \{i \in N : s_i < 0\}$, and the set of creditors as: $C = \{j \in N : s_j > 0\}$ and work with the positive quantities $d_i = -s_i$ for each debtor and $c_j = s_j$ for each creditor.

```
matlab

D = find(saldo < 0);
C = find(saldo > 0);
d = -saldo(D);
c = saldo(C);
```

By construction the balances always sum to zero:

$$\sum_{k \in N} s_k = 0 \Rightarrow \sum_{i \in D} d_i = \sum_{j \in C} c_j$$

meaning the total amount owed by debtors always equals the total amount to be received by creditors. This is the fundamental conservation property that makes exact settlement always possible.

The reason for working in integer cents throughout is both practical and mathematical. Financially, all real transactions happen in whole cents so no precision is lost. Computationally, it eliminates floating point arithmetic errors that can cause solvers to produce slightly

inconsistent results, and it allows the transfer variables to be declared as integers rather than continuous reals, which we return to in Section 4.2.

4.2 Decision variables

Once we have the sets of debtors and creditors and their respective amounts, we need to define what the solver is actually going to decide. The model uses two sets of variables, one for each type of information it needs to track.

The first set are the transfer amount variables. For every possible pair (i, j) where i is a debtor and j is a creditor, we define:

$$x_{ij} \geq 0,\ x_{ij} \in Z\ \forall i \in D,\ j \in C$$

Where x_{ij} represents the amount of money in cents that debtor i pays to creditor j. It is declared as a non-negative integer because we work in whole cents and no transfer can be negative.

The second set are the transaction indicator variables:

$$y_{ij} \in \{0, 1\}\ \forall i \in D,\ j \in C$$

Where $y_{ij} = 1$ means a transaction takes place from debtor i to creditor j, and $y_{ij} = 0$ means no money flows between them. These binary variables are what make this a mixed integer program rather than a standard linear one. Without them there would be no way to count the number of transactions, only the total amount moved.

If there are $p = |D|$ debtors and $q = |C|$ creditors, the model has pq integer variables and pq binary variables, for a total of $2pq$ variables. In the code all variables are stacked into a single vector where the first pq positions correspond x_{ij} and the next pq positions correspond to y_{ij}:

```
matlab
nX = nd*nc;
nY = nd*nc;
nVar = nX + nY;

var_x = 1:nX;
var_y = nX + (1:nY);
```

The properties of each variable are then passed to CPLEX. The objective vector puts zeros on all x_{ij} variables (we do not care about minimizing amounts) and ones on all y_{ij} variables (we are

counting transactions). The lower bound is zero for all variables, the upper bound is infinity for x_{ij} and 1 for y_{ij}, and the type is integer for x_{ij} and binary for y_{ij}:

```
 matlab
obj   = [zeros(nX,1); ones(nY,1)];
lb    = zeros(nVar,1);
ub    = [inf(nX,1); ones(nY,1)];
ctype = [repmat('I',1,nX), repmat('B',1,nY)];
sense = 'minimize';
```

The ctype string tells CPLEX the type of each variable : ‘I' for integer and 'B' for binary. The sense = 'minimize' tells it the direction of the objective.

4.3 Objective function

The goal is to minimize the total number of transactions. Since each transaction corresponds to exactly one binary variable being equal to 1, the objective function is simply:

$$min \sum_{i \in D} \sum_{j \in C} y_{ij}$$

This counts how many of the binary indicators are switched on. The solver will try to find a settlement where as few y_{ij} values equal 1 as possible, while still satisfying all the constraints that ensure every debt is settled exactly. Importantly, this objective says nothing about how much money moves in each transaction, only about how many transactions occur. This is what makes it fundamentally different from minimizing total flow, which is what a standard LP would do.

```
matlab

obj = [ zeros(nX,1); ones(nY,1) ];

sense = 'minimize';
```

4.4 Constraints

Now that the objective is clear, we need to define the rules that make sure the solution is actually valid. The objective alone just tries to turn off as many transactions as possible, but without constraints it would produce meaningless results.

A1: Each debtor pays exactly what they owe

The first requirement is simple. Every debtor must pay their full debt, no more and no less. Debtor i may split their payment across multiple creditors, but the total must add up to exactly their balance d_i. They cannot pay more or less.

$$\sum_{j \in C} x_{ij} = d_i \, \forall i \in D$$

In the code, we build this row by row. Each row corresponds to one debtor.

```
matlab
fila=1; inicio=1;
for i=nc:nc:nX
    A1(fila, inicio:i)=ones(1,nc);
    fila=fila+1; inicio=i+1;
end
lhs1=d(:);
rhs1=d(:);
```

---

each row sums all transfers from debtor i

both left and right hand side are equal to d_i, enforcing equality

A2: Each creditor receives exactly what they are owed

This is the symmetric condition. Every creditor must receive their full amount.

In plain terms: if person j is owed c_j, then the total money they receive from all debtors must equal exactly that.

$$\sum_{i \in D} x_{ij} = c_j \, \forall j \in C$$

This ensures that no creditor is underpaid or overpaid.

```
Matlab

for j=1:1:nc
  for i=1:1:nd
      pos = (i-1)*nc + j;
      A2(j,pos)=1;
  end
end
lhs2=c(:);
rhs2=c(:);
```

A3: Big M linking constraint

This is the most important constraint in the entire model.

So far, we have:

- x_{ij}: how much money is sent
- y_{ij}: whether a transaction exists

But nothing yet connects them.

Without this constraint, the solver could:

- set all $y_{ij} = 0$ (so objective is zero)
- still move money using x_{ij}

Which completely breaks the model.

So we need to enforce:

- if money flows, then the transaction must be active

This is done using the Big M constraint:

$$x_{ij} \leq min(d_i, c_j) \cdot y_{ij} \quad \forall i \in D, j \in C$$

The intuition is that if y_ij = 0, then x_ij is forced to be 0, no money can flow between debtor i and creditor j unless the transaction is active. If y_ij = 1, then x_ij can take any non negative integer value up to the bound min(d_i, c_j). This bound is the tightest one possible, since debtor

i can never transfer more than their total debt d_i, and creditor j can never receive more than their total credit c_j. A generic huge constant M would also work, but choosing the tight bound min(d_i, c_j) gives a much better LP relaxation, which in practice means the solver explores far fewer branch and bound nodes before reaching the integer optimum.

In code, each row of A3 corresponds to one (i, j) pair. We put a 1 on the column of the corresponding x_ij and a -min(d_i, c_j) on the column of the corresponding y_ij. The right hand side is 0, the left hand side is -inf because the constraint is one sided (≤):

matlab

```
k=1; z=1;
for i=1:nY
    A3(i,i)=1;
    A3(i,var_y(i))=-min(d(k),c(z));
    if z==nc
        k=k+1; z=0;
    end
    z=z+1;
end
lhs3=-inf(nX,1);
rhs3=zeros(nY,1);
```

The two index counters k and z walk through the (debtor, creditor) pairs in the same order as the variables were stacked, so the constraint matrix lines up with the variable vector exactly.

4.5 Complete base model summary

With the variables, objective and constraints all in place, we can write the base model compactly as a single block:

$$min \sum_{i \in D} \sum_{j \in C} y_{ij}$$

subject to

$$\sum_{j \in C} x_{ij} = d_i \quad \forall\, i \in D \qquad (A1)$$

$$\sum_{i \in D} x_{ij} = c_j \quad \forall\, j \in C \qquad (A2)$$

$$x_{ij} \leq min(d_i,\, c_j) \cdot y_{ij} \quad \forall\, i \in D,\, j \in C \qquad (A3)$$

$$x_{ij} \in \mathbb{Z}_{\geq 0} \quad \forall\, i \in D,\, j \in C$$

$$y_{ij} \in \{0,\, 1\} \quad \forall\, i \in D,\, j \in C$$

Letting p = |D| and q = |C|, the model has 2pq variables (pq integer transfer amounts and pq binary indicators) and pq + p + q constraints. Our toy example with p = 3 and q = 2 therefore has 12 variables and 11 constraints, which CPLEX cracks in a fraction of a second. As we will see in section 7 the size grows quadratically with the group, and a 17 person instance with roughly 8 debtors and 9 creditors gives 144 variables and around 161 constraints, which already takes the solver close to a minute on average.

Once all the matrices are stacked and passed to CPLEX, solving and extracting the transactions is straightforward:

matlab

```
A   = [A1; A2; A3];
lhs = [lhs1; lhs2; lhs3];
rhs = [rhs1; rhs2; rhs3];

cplex = Cplex('Liquidacion_Gastos');
cplex.Model.sense = sense;
cplex.Model.obj   = obj;
cplex.Model.lb    = lb;
cplex.Model.ub    = ub;
cplex.Model.ctype = ctype;
cplex.Model.A     = A;
cplex.Model.lhs   = lhs;
cplex.Model.rhs   = rhs;

cplex.solve();
sol   = cplex.Solution.x;
x_sol = sol(var_x);
y_sol = sol(var_y);
```

The solution vector contains the optimal x and y values, which we then scan to read off the actual transactions, converting cents back to euros for display.

4.6 Problem with the base model and motivation for A4

The base model finds the optimum, but the optimum is not unique. There are usually many ways to settle debts using the same minimum number of transactions, and the solver picks whichever feasible point it converges to first. In practice this can produce solutions that are mathematically optimal but socially awkward.

To illustrate, when running the base model on a 10 person random instance, person 6 ended up making 4 separate payments while several other debtors made none at all. The total transaction count was minimal, just as the model promised, but the workload was concentrated on a single person. From the solver's point of view "person A makes 4 transfers" and "four different people make 1 transfer each" are both four transactions and therefore equally good. From a user's

point of view they are not. Opening your banking app four times to send four payments is much less appealing than opening it once.

This is what motivates a fourth constraint. Rather than just minimizing the total number of transactions, we want to also bound how many transfers any single debtor is forced to perform. By adding a per debtor cap we prevent the optimum from offloading the entire workload onto one or two unlucky participants, and we end up with a settlement that is fair both globally (the same total number of transactions) and locally (no one has to send too many payments).

We call this constraint A4 and it takes the form of a transfer cap parameter K. Choosing K is a small design decision, and we discuss it at the end of section 4.7 once the constraint is in place.

4.7 Extended model with A4

A4: maximum K transfers per debtor

The constraint reads:

$$\sum_{j \in C} y_{ij} \leq K \qquad \forall\, i \in D \qquad (A4)$$

In words, each debtor can have at most K active transactions in the final solution. Since the y_ij are binary, summing them just counts how many of debtor i's potential transfers are switched on, and the constraint caps that count.

In code, A4 is built one row per debtor. Each row has a 1 on the y_ij columns belonging to that debtor and zeros everywhere else. The right hand side is K and the left hand side is -inf since the constraint is one sided:

matlab

```
K = 3;

for i = 1:nd
    for j = 1:nc
        pos = (i-1)*nc + j;
        A4(i, var_y(pos)) = 1;
    end
end
lhs4 = -inf(nd,1);
rhs4 = K*ones(nd,1);
```

Choosing K.

The natural question is what value of K is reasonable. If K is too small the model becomes infeasible, since with very few allowed transfers per debtor it may not be possible to clear all debts at all. If K is too large the constraint adds nothing and we are back to the base model.

To find a good value we ran a battery of experiments (described in detail in section 7) sweeping through K = 2 and K = 3 across group sizes from 6 to 17 people with 5 random seeds each, for a total of 35 instances. The findings were clean.

K = 2 is too tight. Out of 35 tested instances, 9 were infeasible (around 26% overall, with the rate spiking close to 40% on certain group sizes). The model simply could not always settle the group with at most two transfers per debtor.

K = 3 is the sweet spot. 34 out of 35 instances were feasible, the only exception being a corner case with two debtors and four creditors where the geometry of the problem leaves no slack. Better still, in every feasible case the total number of transactions was identical to what the base model produced. Adding A4 with K = 3 came at no cost to the optimum.

Based on this we use K = 3 throughout the rest of the report. A real shared expense app would likely set this number based on user feedback, but from a pure optimization point of view K = 3 captures essentially all the benefit of the extension without sacrificing solution quality.

4.8 Complete extended model summary

Putting it all together, the extended model is:

$$min \sum_{i \in D} \sum_{j \in C} y_{ij}$$

subject to

$$\sum_{j \in C} x_{ij} = d_i \quad \forall\, i \in D \quad (A1)$$

$$\sum_{i \in D} x_{ij} = c_j \quad \forall\, j \in C \quad (A2)$$

$$x_{ij} \leq min(d_i,\, c_j) \cdot y_{ij} \quad \forall\, i \in D,\, j \in C \quad (A3)$$

$$\sum_{j \in C} y_{ij} \leq K \quad \forall\, i \in D \quad (A4)$$

$$x_{ij} \in \mathbb{Z}_{\geq 0} \quad \forall\, i \in D,\, j \in C$$

$$y_{ij} \in \{0,\, 1\} \quad \forall\, i \in D,\, j \in C$$

Compared to the base model the only structural change is the addition of p extra rows to the constraint matrix, one per debtor, giving a total of pq + 2p + q constraints. The variable count is unchanged at 2pq. From the solver's perspective the impact is small. The LP relaxation tightens

slightly, the search tree may take a few more nodes to explore, and the overall solve time stays in the same order of magnitude as the base model.

In implementation terms the change is just two extra code blocks (the ones shown above for A4) and one extra parameter K. Our wrapper function resolver_liquidacion takes a flag usarLimite that toggles A4 on or off, which is what we use in the experiments to compare both versions side by side on identical inputs.

5: Worked example

To make all this concrete, we walk through the toy example from the introduction with gastos = [45 5 15 35 0 20], where the entries are each person's total expenditure in cents.

Step 1, balances.
The total expenditure is 120 cents. With 6 participants the fair share is 20 each, with no remainder to distribute. The vector of balances is then:

$$saldo \;=\; gastos \;-\; reparto \;=\; [25,\; -15,\; -5,\; 15,\; -20,\; 0]$$

So persons 1 and 4 are creditors (positive balance), persons 2, 3 and 5 are debtors (negative balance), and person 6 is exactly even and drops out of the optimization entirely. Concretely:

$$D \;=\; \{2,\, 3,\, 5\} \qquad d \;=\; (15,\, 5,\, 20)$$
$$C \;=\; \{1,\, 4\} \qquad c \;=\; (25,\, 15)$$

with totals checking out, since the debt total is 15 + 5 + 20 = 40 and the credit total is 25 + 15 = 40. As expected, balances always sum to zero.

Step 2, problem dimensions.
With p = 3 debtors and q = 2 creditors we have 6 transfer amount variables (x_{2,1}, x_{2,4}, x_{3,1}, x_{3,4}, x_{5,1}, x_{5,4}) and 6 transaction indicator variables, for a total of 12 variables. The constraint matrix has 3 rows for A1, 2 rows for A2 and 6 rows for A3, so 11 rows in total. This is a tiny instance and CPLEX solves it in around 30 milliseconds.

Step 3, solving.
Feeding the model to CPLEX gives the optimal objective value:

$$objective \;=\; 3$$

with the following set of transactions:

Deudor	Acreedor	Importe (€)
2	4	0.15
3	1	0.05
5	1	0.20

Three transactions, and everyone is paid out exactly. Person 5 pays the biggest creditor (person 1) directly, person 3 pays person 1 the small remainder of 5, and person 2 pays the smaller creditor (person 4) in full. The solver verified A1 (each debtor pays their total: 15, 5, 20), A2 (each creditor receives their total: 25 = 5 + 20 and 15) and A3 (every nonzero transfer has its corresponding y switched on).

Comparison with greedy.
Walking the largest first greedy heuristic by hand on this instance also lands on 3 transactions: P5 (owing 20) pairs with P1 (owed 25), clearing P5 and leaving P1 needing 5; P2 (owing 15) pairs with P4 (owed 15), clearing both; and P3 (owing 5) pays the remaining 5 to P1. So in principle the textbook greedy heuristic agrees with our MILP here. What is more interesting, and shown in the next subsection, is that the actual Tricount app does not. It returns 4 transactions on the same input, which means the heuristic Tricount uses internally is not pure largest first greedy.

This is the central claim of the report made concrete on the smallest possible example: even with six people and 120 cents, an industry app gives up the optimum, and the MILP recovers it.

5.1 Comparison with the actual Tricount app

To round this off we plugged the same toy input into the real Tricount app and let it produce its own settlement plan. Each person was added with their corresponding expense (P1 = 0.45, P2 = 0.05, P3 = 0.15, P4 = 0.35, P5 = 0.00, P6 = 0.20) and the app was asked to compute the balances and the suggested reimbursements directly. The screenshots below show the three relevant views from the app.

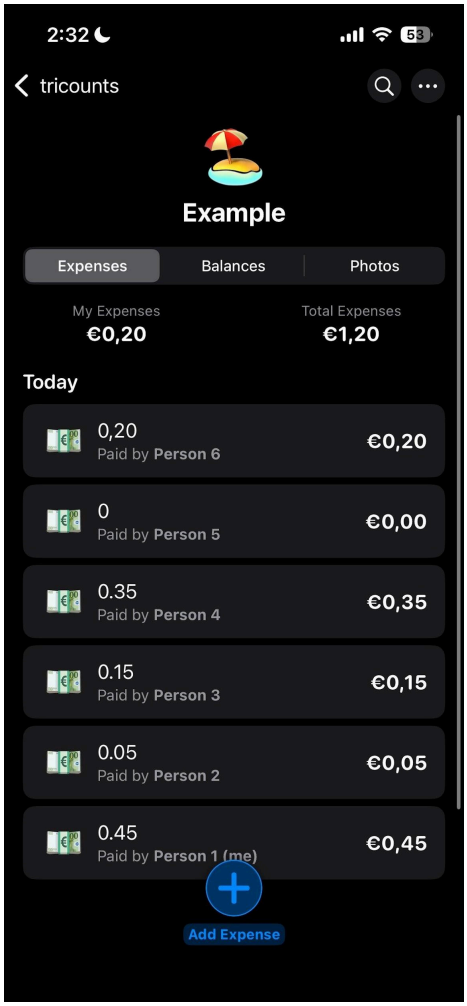


Figure: Expenses view in Tricount with the toy input.

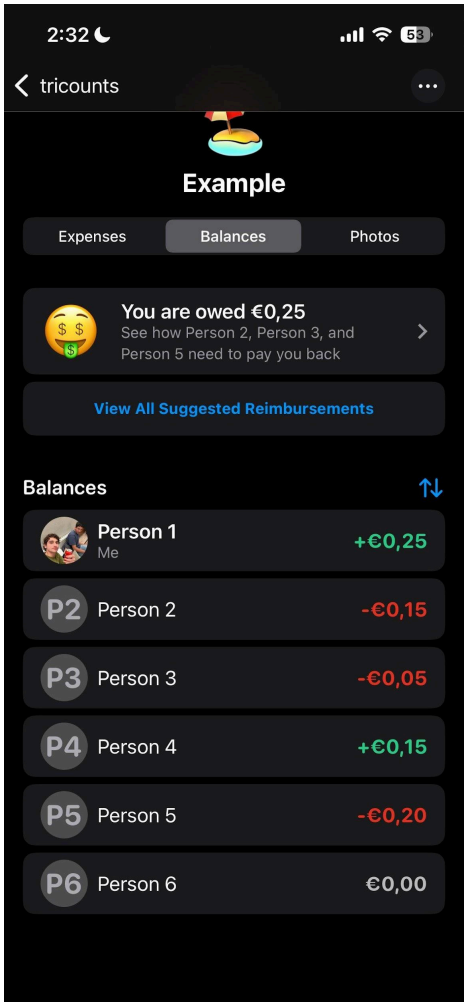


Figure: Net balances Tricount computes from the expenses. They match exactly the saldo vector we computed manually in section 5.

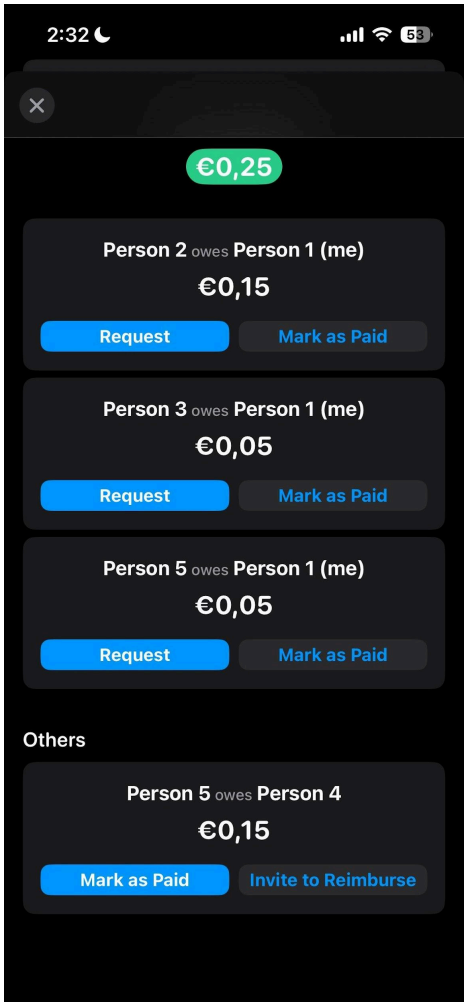


Figure: Suggested reimbursements proposed by Tricount. Four transactions in total.

The settlement Tricount proposes is: P2 → P1 €0.15, P3 → P1 €0.05, P5 → P1 €0.05 and P5 → P4 €0.15. That is four transactions, not three. Our MILP solves the same instance with three (P2 → P4 €0.15, P3 → P1 €0.05, P5 → P1 €0.20), so on this exact toy example the MILP saves one transaction over what the actual app produces.

What is interesting is that, as we showed above, even a manually applied largest first greedy heuristic also finds 3 transactions on this instance. So Tricount is not running pure largest first greedy under the hood. Whatever heuristic they use prefers to split P5's payment across two creditors and consolidate receipts on P1, which produces a valid settlement but adds one extra transaction. The takeaway is that the gap between MILP and a real shared expense app shows up immediately, on the smallest possible example. As section 7 will confirm with the random instances, the gap is real and can grow with group size.

6: Comparison with alternative formulations

The MILP we have built is one way to model debt settlement, but not the only one. Here we compare it with three alternatives, each making different trade offs.

6.1 Greedy heuristic (the Tricount approach)

Already discussed in detail in section 2. The key contrast is what each method optimizes. Greedy minimizes the largest outstanding debt at each step, our MILP minimizes the total count of transactions over the whole settlement. These are different objectives that happen to coincide on simple instances and diverge on complex ones.

Greedy runs in O(n^2) and produces a solution in milliseconds on any group. Our MILP can take seconds to minutes on groups of 15 or more, but guarantees the optimal answer. Both Mithun Mohan K (2019) and the Coding Blocks article describe variants of greedy as the core of how Splitwise actually works. Their reasoning is operational. At the scale of millions of users, the difference between an optimal solution and a near optimal one is not worth the computational cost.

6.2 Pure linear programming (no integer variables)

A natural question is whether we can avoid the integer constraints entirely and solve the problem as a standard LP. The answer is no, but the reason is instructive.

If we drop the binary variables y_ij and try to minimize, say, the total amount of money moved Σ x_ij, the LP gives us a minimum cost flow solution. This finds the cheapest way to route money, but cheap is not what we want. We want fewest transactions, regardless of how much each one carries. The LP has no way to express "count transactions" because counting requires distinguishing zero from non zero, and a continuous variable cannot do that.

You can try to tweak the objective with weights, but no continuous penalty term ever forces an x_ij to be exactly zero unless it is already optimal to set it that way. The combinatorial nature of "use this connection or not" is intrinsically discrete, and that is exactly what the binary y_ij variables capture.

6.3 Quadratic programming

Another approach is to use a quadratic objective. For example, minimize Σ x_ij^2, which penalizes large transfers and tends to spread payments out across many small ones. This is sometimes proposed as a way to make settlements feel "fairer" in the sense that no one pays a single huge amount.

The problem is that this solves a different problem. Spreading payments out increases the number of transactions, which is the opposite of what we want. The QP formulation also keeps the model continuous and skips the integer programming machinery, which is a feature in some applications but a bug here. There is no quadratic objective that minimizes a count, since counts are inherently combinatorial and cannot be encoded with a smooth quadratic term.

6.4 MILP with Small-O linking instead of Big-M

The most interesting comparison is with the MILP described in Alex Brou's Medium article. The structure is essentially the same as ours: integer transfer variables, binary indicators, the same A1 and A2 constraints. The difference is in how x_ij and y_ij are linked.

Our model uses the Big-M form:

$$x_{ij} \leq M_{ij} \cdot y_{ij}, \quad with\ M_{ij} = min(d_i, c_j)$$

which forces x_ij to zero whenever y_ij is zero. Brou's article uses what we call a Small-O form, roughly:

$$y_{ij} \geq \varepsilon \cdot x_{ij}, \quad with\ \varepsilon\ a\ small\ positive\ constant$$

Optimality. The MILP always finds a settlement with the minimum possible number of transactions, while greedy heuristics can use more than the minimum in adversarial instances. We confirmed this empirically on the toy example itself. When we ran the same six person input through the actual Tricount app, the app returned 4 transactions while our MILP returned 3, a difference visible on the smallest possible group. The gap shows up as soon as the structure of balances stops aligning with whatever internal heuristic the app applies.

The Big-M form does not have this issue. Choosing M = min(d_i, c_j) is probably the tightest valid bound (no transfer can ever exceed it), which both keeps the model correct and gives the best possible LP relaxation. This is the standard textbook approach to encoding indicator variables in MILP, and it is what we use.

7: Conclusions

We set out to model the debt settlement problem as a mixed integer linear program and to compare the result with the greedy approach used by real shared expense apps. The MILP formulation, with two sets of variables (integer transfer amounts x_ij and binary transaction indicators y_ij), three core constraints (A1, A2, A3), and an optional fairness constraint (A4), produces provably optimal settlements for groups of up to about 17 people in reasonable time.

To validate the model in practice we ran a battery of experiments across group sizes [6, 8, 10, 12, 14, 16, 17] with 5 random seeds each, comparing the base model and the extended model with K = 2 and K = 3. The main findings are the following.

Optimality. The MILP always finds a settlement with the minimum possible number of transactions, while greedy can use more than the minimum in adversarial instances. The toy example where greedy and MILP agree is the easy case, the gap shows up as soon as the structure of balances stops aligning with the largest first heuristic.

Solve time. Performance grows sharply once the group exceeds about 12 people. From around 30 ms at 6 participants to roughly 88 seconds on average at 17, with the worst single instance taking close to 3 minutes. This is consistent with the NP hard nature of the problem, but for the group sizes shared expense apps actually deal with, it remains tractable.

Adding A4 with K = 3 is essentially free. Across the 35 random instances, the extended model found a feasible settlement in 34 of them, and in every feasible case the total number of transactions was identical to the base model. The maximum number of transfers per debtor dropped from an average around 3.4 (with outliers up to 6) to at most 3. K = 2 was too tight, becoming infeasible in roughly a quarter of cases.

Comparison with alternatives. Of the four formulations we considered (greedy, pure LP, quadratic programming, MILP with Big-M), the MILP with Big-M is the only one that actually solves the right problem. Greedy approximates, LP and QP optimize the wrong objective, and the Small-O variant of MILP works in practice but lacks the theoretical grounding of the Big-M form.

Beyond the technical content, what this project showed us is that even a problem as everyday as splitting expenses with friends has a non trivial mathematical core. Our model would not be deployed in a real app, which has to handle millions of users and care about milliseconds, but it would be perfectly suitable for a small group that wants the best possible answer and is willing to wait a few seconds. More importantly, it gave us a chance to take an optimization technique covered in class and apply it end to end on a real, recognizable problem, which is what makes it satisfying.

Bibliography:

https://medium.com/@mithunmk93/algorithm-behind-splitwises-debt-simplification-feature-8ac485e97688

https://www.linkedin.com/pulse/how-does-splitwise-algorithm-work-codingblocksindia-9hhyc

https://medium.com/@alexbrou/split-bills-with-friends-the-algorithm-behind-tricount-and-splitwise-using-integer-programming-48cd01999507